

GROUPE ISM

TRAVAUX PRATIQUES — ÉTUDES DE CAS

ARCHITECTURE DES APPLICATIONS

Du client/serveur aux microservices

Six projets d'étude de cas

Projet 1 — Système d'information d'une clinique privée
Projet 2 — Plateforme de commerce électronique
Projet 3 — Application de microfinance et services financiers
Projet 4 — Plateforme de livraison et suivi de colis
Projet 5 — Système de réservation multi-hôtels
Projet 6 — Portail de services administratifs en ligne

Document pédagogique — Études de conception et d'argumentation technique

Présentation générale

Ces études de cas ont pour objectif de placer les étudiants dans une situation de conception proche d'un projet réel. Chaque groupe reçoit un contexte métier différent et doit proposer une architecture applicative adaptée aux contraintes annoncées.

Le travail attendu ne consiste pas à reproduire une architecture type. Les choix doivent être justifiés à partir du nombre d'utilisateurs, du mode d'accès, de la sensibilité des données, de la disponibilité attendue, de l'évolutivité et des conditions d'exploitation.

Compétences mobilisées

- Identifier les clients, serveurs, services, données et flux d'une application.
- Choisir entre une architecture 2-tiers et 3-tiers et justifier ce choix.
- Distinguer les tiers de déploiement et les couches logiques internes de l'application.
- Évaluer la pertinence d'un monolithe ou d'un découpage en microservices.
- Définir des API et lire des échanges HTTP/HTTPS utilisant JSON et des mécanismes d'autorisation.
- Proposer un mode de déploiement cohérent : serveur physique, VM et, lorsque cela est pertinent, conteneurs Docker.
- Prendre en compte sécurité, disponibilité, supervision, journalisation, sauvegarde et évolutivité.

Consignes de production

1. Présenter le besoin et les contraintes avant de proposer une solution.
2. Produire au minimum un diagramme logique de l'application et un diagramme de déploiement ou d'architecture en tiers.
3. Nommer clairement les composants, les responsabilités et les flux. Les diagrammes doivent être lisibles et réalisés avec un outil adapté comme draw.io, Visio ou équivalent.
4. Justifier les choix d'architecture. Une réponse non argumentée est considérée incomplète.
5. Pour les API, proposer des URI cohérentes, les méthodes HTTP utiles et au moins un exemple de requête ou de réponse JSON.
6. Préciser les principaux mécanismes de sécurité et d'exploitation associés au projet.
7. Préparer une courte soutenance permettant de défendre les choix retenus et de répondre aux questions techniques.

Livrable commun

Format	Document PDF de 5 à 8 pages par groupe, annexes techniques non comprises.

Diagrammes	Architecture logique + architecture en tiers ou déploiement.
API	Table des principaux endpoints et un exemple HTTP/JSON.
Argumentation	Justification des choix 2-tiers/3-tiers, couches, monolithe/microservices et déploiement.

PROJET 1

Système d'information d'une clinique privée

Dossiers patients, consultations, rendez-vous, prescriptions et facturation

Fiche du projet

Domaine	Santé / gestion Clinique
Utilisateurs	Médecins, secrétaires, infirmiers, comptabilité et administration
Canaux	Postes internes ; accès distant réservé à certains profils
Données	Dossiers patients, rendez-vous, consultations, prescriptions, factures
Enjeu dominant	Confidentialité, contrôle d'accès et traçabilité
Niveau attendu	Intermédiaire

Contexte

Une clinique privée exploite aujourd'hui plusieurs fichiers et petites applications indépendantes. La direction souhaite mettre en place une application unique permettant de gérer les patients, les rendez-vous, les consultations, les prescriptions et la facturation.

Le personnel administratif travaille principalement à l'accueil et au service de facturation. Les médecins doivent consulter les dossiers depuis différentes salles. Certains responsables souhaitent également disposer d'un accès distant sécurisé pour consulter des informations de suivi.

Le système doit limiter l'accès aux données selon le rôle : une secrétaire ne doit pas disposer des mêmes droits qu'un médecin, et les actions importantes doivent pouvoir être retracées.

Contraintes à prendre en compte

- Environ 80 utilisateurs internes avec plusieurs connexions simultanées.
- Données sensibles nécessitant authentification, autorisation et journalisation.
- Continuité de service attendue pendant les heures d'ouverture.
- Accès distant autorisé uniquement pour certains profils.
- Sauvegarde quotidienne et possibilité de restaurer les dossiers.
- L'équipe informatique reste de taille réduite : l'architecture doit rester exploitable.

Mission confiée au groupe

Concevoir une architecture applicative cohérente pour la clinique en privilégiant la sécurité et la simplicité d'exploitation. Le groupe doit décider si une architecture 2-tiers reste acceptable ou si le 3-tiers est plus adapté, puis organiser les principales fonctions en couches et définir les API utiles.

Travail demandé

Élément à produire	Attendu	Justification à fournir
Choix 2-tiers / 3-tiers	Architecture retenue et position des trois responsabilités principales.	Justifier par les accès, la sécurité et la maintenance.
Architecture logique	Présentation, services métier, accès aux données, services transverses.	Expliquer la responsabilité de chaque bloc.
Modules	Patients, rendez-vous, consultations, prescriptions, facturation, utilisateurs.	Préciser les dépendances entre modules.
API	Endpoints essentiels pour patients, rendez-vous et consultations.	Définir méthodes HTTP et règles d'accès.
Déploiement	Serveur(s), VM ou autre organisation adaptée.	Justifier l'isolation et la simplicité d'exploitation.
Sécurité	Authentification, rôles, HTTPS, logs, sauvegardes.	Relier chaque mécanisme à un risque concret.

Scénario à analyser

- Un médecin ouvre le dossier d'un patient depuis un poste de consultation.
- Le système vérifie son identité et ses droits.

- Le médecin ajoute une observation et une prescription.
- La modification doit être enregistrée et tracée.
- Le groupe doit décrire le flux complet entre interface, logique métier et données.

Décision d'architecture

Le groupe doit expliquer si le client doit accéder directement à la base de données. Si le 3-tiers est retenu, préciser le rôle du serveur applicatif dans le contrôle des droits et la protection des données.

Organisation en couches

- Couche présentation : écrans de consultation, recherche, saisie.
- Couche métier : règles de rendez-vous, validation d'une consultation, facturation.
- Couche accès aux données : lecture et écriture dans le SGBD.
- Couche données : patients, actes, prescriptions, utilisateurs, factures.

Exemple d'échange API à compléter ou adapter

```
GET /api/patients/125 HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>
Réponse attendue : 200 OK + dossier autorisé, ou 403 Forbidden si le rôle ne permet pas l'accès.
```

Points de vigilance

- Ne pas exposer directement la base aux postes utilisateurs.
- Distinguer authentification et autorisation.
- Prévoir la traçabilité des consultations et modifications sensibles.
- Éviter une architecture microservices injustifiée pour une équipe d'exploitation réduite.

Livrables attendus

- Diagramme logique de l'application clinique.
- Diagramme 3-tiers ou justification argumentée d'un autre modèle.
- Tableau des rôles et droits principaux.
- Liste de 8 à 12 endpoints API proposés.
- Proposition de déploiement et de sauvegarde.
- Note de décision : monolithe modulaire ou microservices, avec justification.

Questions pour la soutenance

1. Pourquoi le navigateur ou le poste médical ne devrait-il pas accéder directement à la base ?
2. Quelle différence faites-vous entre le rôle « médecin » et l'authentification de l'utilisateur ?

3. Pourquoi un monolithe bien structuré peut-il être pertinent ici ?
4. Quel composant doit être journalisé en priorité et pourquoi ?

Attendu pédagogique

Le groupe doit défendre une architecture cohérente avec le besoin. Une solution plus complexe n’est pas considérée meilleure si elle n’est pas justifiée par la charge, la sécurité, la disponibilité, l’exploitation ou l’évolutivité.

Binôme : Aichatou Ka – Fatima Zahra Diop.

Classe : L3 IA.

Approche architecturale : Partir des contraintes pour construire la solution

« Avant de choisir l'architecture, il faut partir des contraintes du système et du besoin métier plutôt que de la technologie. Dans cette clinique, plusieurs profils d'utilisateurs accèdent à des données médicales sensibles, mais ils n'ont ni les mêmes droits ni les mêmes responsabilités. Nous avons également des exigences de confidentialité et de traçabilité, ainsi qu'un besoin d'accès distant pour certains rôles.

L'architecture doit donc permettre à chaque utilisateur d'accéder aux fonctionnalités dont il a besoin, sans lui donner un accès direct aux données ni au SGBD. **La question est alors de déterminer où placer le contrôle des accès, des règles métier et des opérations sur les données. »**

L'architecture 2-tiers est-elle encore acceptable ou faut-il retenir une architecture 3-tiers ?

Nous retenons définitivement une architecture 3-tiers pour le système d'information de la clinique.

Ce choix découle directement des contraintes du projet. La clinique manipule des **données médicales sensibles**, doit gérer plusieurs catégories d'utilisateurs disposant de **droits différents**, assurer la **traçabilité des actions importantes** et permettre un **accès distant réservé à certains profils**. Le système doit également supporter environ 80 utilisateurs internes avec plusieurs connexions simultanées, assurer une continuité de service et rester exploitable par une équipe informatique réduite.

Une architecture **2-tiers** reste techniquement envisageable, mais elle est moins adaptée à ce contexte, notamment parce que le projet impose de ne pas exposer directement le SGBD aux postes utilisateurs. Elle rendrait également moins naturelle la centralisation des contrôles d'accès et des règles métier.

Nous retenons donc une architecture **3-tiers**, organisée selon trois responsabilités principales :

1. **Présentation** : interfaces utilisées par les médecins, secrétaires, infirmiers, responsables et autres profils autorisés ;
2. **Traitement applicatif** : serveur applicatif chargé d'appliquer les règles métier, les contrôles d'accès et les traitements fonctionnels ;
3. **Données** : SGBD contenant les dossiers patients, rendez-vous, consultations, prescriptions, utilisateurs et factures.

Le **serveur applicatif constitue la frontière de contrôle entre les utilisateurs et les données**. Les requêtes provenant des clients passent par cette couche, qui vérifie les droits, applique les

règles métier et contrôle l'accès au SGBD. La base de données n'est donc pas directement accessible depuis les postes utilisateurs, conformément aux exigences du projet.

Cette séparation améliore également la **maintenabilité** en isolant les responsabilités de présentation, de traitement et de données. Une évolution de l'interface ou d'une règle métier peut ainsi être réalisée sans créer une dépendance directe entre le client et le SGBD.

Enfin, afin de conserver une architecture adaptée à la taille de l'équipe informatique, le **serveur applicatif sera conçu comme un monolithe modulaire** plutôt que comme un ensemble de microservices. Cette décision évite d'introduire une complexité d'exploitation non justifiée par les besoins actuels, tout en conservant une organisation interne claire et évolutive. Le projet demande en effet de privilégier une architecture cohérente avec le besoin et la simplicité d'exploitation.

Décision finale : une architecture 3-tiers avec un serveur applicatif centralisé, organisé en monolithe modulaire, est retenue comme le meilleur compromis pour répondre aux exigences d'accès, de sécurité et de maintenance de la clinique.

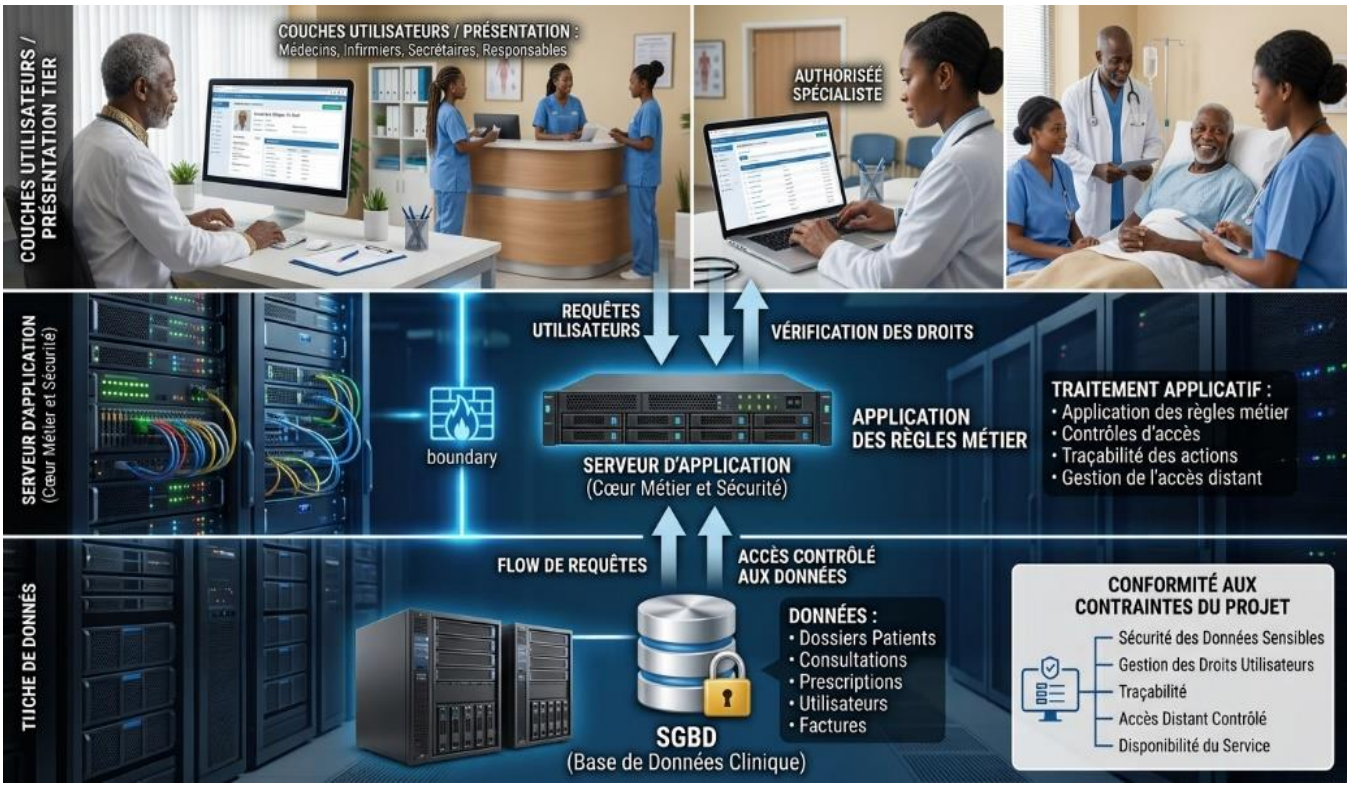
Architecture logique : présentation, services métier, accès aux données, services transverses. Expliquer la responsabilité de chaque bloc.

Architecture logique

L'architecture logique est organisée autour de quatre blocs : **présentation, services métier, accès aux données et services transverses**.

- **Présentation :** ce bloc assure l'interaction avec les utilisateurs et transmet leurs demandes à l'application.
- **Services métier :** ce bloc regroupe les traitements et règles métier de l'application.
- **Accès aux données :** ce bloc assure les échanges entre les services métier et le SGBD, notamment pour les opérations de lecture et d'écriture.
- **Services transverses :** ce bloc fournit les mécanismes communs utilisés par les différents composants, notamment l'authentification, l'autorisation et la journalisation/traçabilité, éléments importants compte tenu de la sensibilité des données et des exigences du projet.

Les dépendances principales suivent le flux **présentation → services métier → accès aux données → SGBD**. Les services transverses sont utilisés par les différents blocs selon leurs besoins.



Modules : Patients, rendez-vous, consultations, prescriptions, facturation, utilisateurs. Préciser les dépendances entre modules.

L'application est organisée autour de six modules fonctionnels : **Patients, Rendez-vous, Consultations, Prescriptions, Facturation et Utilisateurs**.

Le module **Patients** constitue le référentiel des dossiers patients. Le module **Rendez-vous** s'appuie sur les patients pour organiser les rendez-vous. Le module **Consultations** s'appuie sur le patient et, lorsqu'il y a lieu, sur le rendez-vous afin d'enregistrer les actes et informations médicales. Une **prescription** est rattachée à une consultation et est créée par un utilisateur disposant des droits nécessaires. La **facturation** s'appuie notamment sur les actes ou prestations associés aux consultations.

Le module **Utilisateurs** est transversal : il gère les identités et les rôles permettant de contrôler les opérations autorisées dans les différents modules. Cette séparation respecte le besoin de différencier les droits des médecins, secrétaires, infirmiers, personnels de comptabilité et administrateurs.

Les principales dépendances fonctionnelles sont donc :

Patients → Rendez-vous → Consultations → Prescriptions

et **Consultations → Facturation**.

Le module **Utilisateurs** intervient transversalement pour la gestion des identités, des rôles et des autorisations.

API : Endpoints essentiels pour patients, rendez-vous et consultations. Définir méthodes HTTP et règles d’accès.

API : Endpoints essentiels pour patients, rendez-vous et consultations

L'API constitue l'interface entre les clients et le serveur applicatif. Elle permet d'accéder aux fonctionnalités du système sans exposer directement le SGBD aux postes utilisateurs. Les endpoints sont définis selon les ressources **patients, rendez-vous et consultations**, avec une méthode HTTP et une règle d'accès associée.

API — Endpoints essentiels et règles d’accès

Système d’information — Clinique privée

RESSOURCE	HTTP	ENDPOINT	RÈGLE D’ACCÈS
Patients	GET	/api/patients	Authentifié + autorisé
Patient	GET	/api/patients/{id}	Permission d'accès au dossier
Patient	POST	/api/patients	Permission de création
Patient	PATCH	/api/patients/{id}	Permission de modification
Rendez-vous	GET	/api/appointments	Authentifié + autorisé
Rendez-vous	POST	/api/appointments	Permission de création
Rendez-vous	PATCH	/api/appointments/{id}	Permission de modification
Consultation	POST	/api/consultations	Professionnel autorisé
Consultation	GET	/api/consultations/{id}	Permission de consultation
Historique consultations	GET	/api/patients/{id}/consultations	Professionnel autorisé

L'accès à l'API repose sur une **authentification** permettant d'identifier l'utilisateur, puis sur une **autorisation** permettant de vérifier que son rôle possède la permission nécessaire. Les accès et modifications sensibles doivent également pouvoir être tracés, conformément aux exigences de confidentialité et de traçabilité du projet.

Déploiement (Serveur(s), VM ou autre organisation adaptée) :

Déploiement de l’architecture

L'architecture est déployée sur une infrastructure virtualisée composée d'un **serveur physique hébergeant un hyperviseur VMware ou Proxmox**. Trois machines virtuelles sont utilisées afin de séparer les principales responsabilités de l'application :

Composant	Responsabilité
VM1 — Serveur Web Présentation	Point d'entrée HTTPS de l'application, gestion des échanges avec les clients et transmission des requêtes au serveur applicatif
VM2 — Serveur Application	API, logique métier, gestion des authentifications, autorisations et traitements fonctionnels
VM3 — Serveur SGBD PostgreSQL	Stockage et gestion des données de la clinique

Le **client n'accède pas directement au SGBD**. Les échanges suivent le flux :

Client → Serveur Web → Serveur Application → PostgreSQL

Un **serveur physique distinct** est consacré au stockage des sauvegardes afin de séparer l'environnement de production des données de sauvegarde.

Cette organisation permet de séparer les responsabilités tout en conservant une infrastructure relativement simple à administrer, ce qui est cohérent avec la présence d'une **petite équipe informatique**.

Limite identifiée : le serveur physique hébergeant les VM constitue un point unique de défaillance. L'architecture ne revendique donc pas une haute disponibilité complète. Les sauvegardes quotidiennes et la procédure de restauration permettent cependant de renforcer la continuité et la capacité de reprise.

Sécurité — authentification, rôles, HTTPS, journalisation et sauvegardes

Authentification :

Risque concret

Si le système ne vérifie pas l'identité des personnes qui accèdent à l'application, des personnes non autorisées pourraient accéder aux données de la clinique, ce qui présenterait notamment un risque de **violation de la confidentialité des données médicales**.

De plus, sans authentification individuelle, il serait difficile d'identifier précisément l'utilisateur à l'origine d'une action ou d'une modification suspecte.

Solution technique

Pour vérifier l'identité d'un utilisateur, le système impose une procédure d'authentification. Chaque membre du personnel dispose d'un **compte individuel** et doit fournir son identifiant et son mot de passe.

Le **serveur applicatif** reçoit la demande d'authentification et vérifie les informations d'identification auprès du système de gestion des utilisateurs avant d'autoriser l'accès à l'application.

Une fois l'utilisateur authentifié, son identité peut être associée aux opérations qu'il réalise, notamment pour assurer leur **traçabilité**.

Les Rôles / Autorisations :

Risque concret

Un utilisateur peut être correctement authentifié tout en n'étant pas autorisé à effectuer certaines opérations. Sans contrôle des droits, il pourrait consulter, modifier ou supprimer des données qui ne relèvent pas de ses responsabilités, compromettant ainsi la confidentialité et surtout l'intégrité des données médicales**.**

Solution technique

Nous mettons en place un **contrôle d'accès basé sur les rôles (RBAC)** au niveau du **serveur applicatif**.

Après authentification, chaque utilisateur est associé à un **rôle**, auquel sont rattachées des **permissions précises**. Pour chaque requête adressée à l'API, le serveur applicatif vérifie :

Identité de l'utilisateur → rôle → permission requise → autorisation ou refus

L'opération n'est exécutée que si la permission correspondante est accordée. Dans le cas contraire, le serveur refuse la requête avec une réponse **HTTP 403 Forbidden**.

Cette approche permet de centraliser l'application des droits d'accès et d'empêcher qu'un utilisateur authentifié puisse contourner les règles métier pour accéder ou modifier des données qui ne lui sont pas autorisées .

Le protocole HTTPS (Port 443) :

Sans chiffrement des communications, des informations sensibles échangées sur le réseau — notamment les identifiants et les données médicales — pourraient être **interceptées et consultées**, particulièrement lors des accès distants.

Solution technique

L'application utilise **HTTPS avec TLS** afin de protéger les communications entre le navigateur et le serveur Web. Le serveur Web écoute sur le **port 443** et utilise un **certificat TLS** valide. Le trafic HTTP non sécurisé sur le port 80 est **redirigé vers HTTPS ou** bloqué selon la politique retenue**.**

« HTTPS protège les données en transit entre le client et le serveur Web contre l'interception sur le réseau. »

La Journalisation (Les Logs) et la Traçabilité :

Sans mécanisme de journalisation, la clinique ne disposerait pas d'une **traçabilité fiable des actions sensibles**. En cas de consultation, modification ou suppression non autorisée d'un dossier patient, il serait difficile d'identifier **l'utilisateur à l'origine de l'action, la date et l'heure, ainsi que l'opération réalisée**. Cela compliquerait l'audit du système et l'analyse d'un incident de sécurité.

Solution technique

Pour chaque **action sensible** traitée par l'API, le **serveur applicatif** génère automatiquement une entrée de journalisation.

Chaque entrée contient notamment :

- **l'identifiant de l'utilisateur** ;
- **son rôle** ;
- **la date et l'heure** de l'action ;
- **le module ou la ressource concernée** ;
- **l'opération effectuée** ;
- **le résultat de l'opération**.

Réponses pour la soutenance :

1. Pourquoi le navigateur ou le poste médical ne devrait-il pas accéder directement à la base ?

R : **R :** Le navigateur ou le poste médical ne doit pas accéder directement à la base afin de ne pas exposer le SGBD aux clients et de centraliser les contrôles d'accès. Dans notre architecture 3-tiers, les requêtes passent par le serveur applicatif, qui contrôle l'authentification, les autorisations et les traitements métier avant d'accéder aux données.

2. Quelle différence faites-vous entre le rôle « médecin » et l'authentification de l'utilisateur ?

R : Tous les utilisateurs y compris le médecin doivent s'authentifier avant d'accéder à l'application tandis que le rôle « médecin » donne aux médecins des droits (écrire modifier des prescriptions, des dossiers médicaux, etc) que les autres profils comme infirmiers ou comptables n'ont pas.

« L'authentification répond à "Qui êtes-vous ?", le rôle et les autorisations répondent à "Qu'avez-vous le droit de faire ?". »

3. Pourquoi un monolithe bien structuré peut-il être pertinent ici ?

R : Un monolithe modulaire est pertinent ici parce qu'il permet de regrouper les fonctionnalités de la clinique dans une application unique tout en séparant clairement les modules fonctionnels. Avec une petite équipe informatique et environ 80 utilisateurs, cette approche permet de conserver une architecture plus simple à déployer, superviser et maintenir qu'une architecture en microservices, qui introduirait davantage de composants et de complexité d'exploitation .

4. Quel composant doit être journalisé en priorité et pourquoi ?

R : Le serveur applicatif est le point prioritaire pour la journalisation des actions métier et de sécurité, car il traite les requêtes de l'utilisateur, applique les contrôles d'accès et connaît l'identité, le rôle, l'opération demandée et son résultat. Les autres composants peuvent également produire leurs propres logs techniques, mais le serveur applicatif est central pour la traçabilité des actions métier.

CONCLUSION

« Notre architecture 3-tiers répond aux besoins de la clinique en combinant sécurité, simplicité et maintenabilité. Le serveur applicatif centralise les contrôles d'accès, les traitements métier et la traçabilité, tandis que le déploiement virtualisé et les sauvegardes renforcent l'exploitation du système. Cette solution reste évolutive tout en étant adaptée au contexte de la clinique. »